

ЛАБОРАТОРНА РОБОТА № 2

Хід роботи:

3.1. Завдання 1: Типізація функцій через generics

Додати generics до функцій з Лабораторної роботи №1, де тип результату залежить від вхідного параметра.

Список функцій: `retryOperation`, `processInBatches`, `raceWithTimeout`.

Вимоги:

- Змінити сигнатури та внутрішні типові анотації — логіку не змінювати.
- Переконавшись, що TypeScript виводить тип результату без явного вказання:
`const result = await retryOperation(op, 3)` — тип `result` має визначитися автоматично.

`retryOperation`:

```
export async function retryOperation<T>(  
  operation: () => Promise<T>,  
  maxRetries: number = 3  
) : Promise<T> {  
  let lastError: unknown;  
  
  for (let attempt = 1; attempt <= maxRetries; attempt++) {  
    console.log(`Спроба ${attempt}...`);  
  
    try {  
      const result = await operation();  
      return result;  
    } catch (error) {  
      lastError = error;  
  
      if (attempt === maxRetries) {  
        break;  
      }  
  
      await new Promise((resolve) => setTimeout(resolve,  
150));  
    }  
  }  
  
  throw lastError;  
}
```

Розроб.	Крамар Д. О.			Звіт з лабораторної роботи	Літ.	Арк.	Аркуші
Перевір.	Фуріхата Д. В.					1	22
Керівник					ФІКТ Гр. ІПЗ-24-3[1]		
Н. контр.							
Зав. каф.							

```
}
```

processInBatches:

```
export async function processInBatches<T, R>(
  items: T[],
  batchSize: number,
  processor: (batch: T[]) => Promise<R[]>
): Promise<R[]> {
  const results: R[] = [];
  const totalBatches = Math.ceil(items.length / batchSize);

  for (let i = 0; i < totalBatches; i++) {
    const currentBatchNumber = i + 1;
    console.log(`Обробка партії ${currentBatchNumber}/${total-
Batches}...`);

    const batch = items.slice(i * batchSize, (i + 1) * batch-
Size);

    const processedBatch = await processor(batch);

    results.push(...processedBatch);
  }

  return results;
}
```

raceWithTimeout:

```
export function raceWithTimeout<T>(
  promise: Promise<T>,
  timeoutMs: number
): Promise<T> {
  const timeoutPromise = new Promise<never>((_ , reject) => {
    setTimeout(() => {
      const err = new Error(`Operation timed out after
${timeoutMs}ms`);
      (err as any).timeoutMs = timeoutMs;
    }, timeoutMs);
  });
  return Promise.race([promise, timeoutPromise]);
}
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – ЛрЗ	Арк.
		Фуріхата Д. В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        reject(err);
    }, timeoutMs);
});

return Promise.race([promise, timeoutPromise]);
}

```

3.2. Завдання 2: Налаштування проєкту для тестування

Налаштувати середовище для запуску Jest-тестів з TypeScript для функцій з Лабораторної роботи №1.

Вимоги:

- Встановити залежності: `npm install -D jest ts-jest @types/jest`.
- Створити `jest.config.js` з preset `ts-jest` та `testEnvironment: 'node'`.

```

module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
};

```

- Додати до `package.json` скрипти: `"test"`, `"test:watch"`, `"test:coverage"`.

```

"scripts": {
  "test": "jest",
  "test:watch": "jest --watch",
  "test:coverage": "jest --coverage",
  "build": "tsc"
},

```

- Переконайтеся, що команда `npm test` запускається без помилок.

3.3. Завдання 3: Тести для `fetchUserProfiles`

```

import { delay } from './t3';
import { fetchUserProfiles, getUserById } from './t4';
import { retryOperation } from './t5';
import { processInBatches } from './t6';
import { raceWithTimeout } from './t7';

describe('fetchUserProfiles()', () => {
  beforeEach(() => {
    jest.useFakeTimers();
  });
});

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр3	Арк.
		Фуріхата Д. В.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    });

    afterEach(() => {
        jest.useRealTimers();
    });

    it('should fetch user profiles for given user IDs', async ()
=> {
        const userIds = ['1', '2'];

        const fetchPromise = fetchUserProfiles(userIds);
        await jest.advanceTimersByTimeAsync(200);

        const profiles = await fetchPromise;

        expect(profiles).toEqual([
            { id: '1', name: 'User 1', email: 'user_1@test.ua' },
            { id: '2', name: 'User 2', email: 'user_2@test.ua' }
        ]);
    });

    it('if userIds is empty, should return an empty array', async
() => {
        const profiles = await fetchUserProfiles([]);

        expect(profiles).toEqual([]);
    });

    it('should return correct object structure and properties',
async () => {
        const fetchPromise = fetchUserProfiles(['99']);

        await jest.advanceTimersByTimeAsync(200);
        const profiles = await fetchPromise;

        expect(profiles).toHaveLength(1);

        expect(profiles[0]).toHaveProperty('id', '99');
        expect(profiles[0]).toHaveProperty('name', 'User 99');
    });

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – ЛрЗ	Арк.
		Фуріхата Д. В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        expect(profiles[0]).toHaveProperty('email',
'user_99@test.ua');
    });
});

```

```

PASS src/fetchUserProfiles.test.ts
  fetchUserProfiles()
    ✓ should fetch user profiles for given user IDs (15 ms)
    ✓ if userIds is empty, should return an empty array (1 ms)
    ✓ should return correct object structure and properties (4 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        0.55 s, estimated 1 s

```

3.4. Завдання 4: Тести для retryOperation

```

import { processInBatches } from './t6';

async function numberProcessor(batch: number[]): Promise<number[]> {
  return batch.map(num => num * 2);
}

async function stringProcessor(batch: string[]): Promise<string[]> {
  return batch;
}

describe('processInBatches()', () => {
  let consoleSpy: jest.SpyInstance;

  beforeEach(() => {
    jest.useFakeTimers();
    consoleSpy = jest.spyOn(console, 'log').mockImplementation(() => {});
  });

  afterEach(() => {
    jest.useRealTimers();
    consoleSpy.mockRestore();
  });
});

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр3	Арк.
		Фуріхата Д. В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

it("should return a correct array of results", async () => {
    const items = [1, 2, 3, 4, 5];

    const result = await processInBatches(items, 2, numberProcessor);

    expect(result).toEqual([2, 4, 6, 8, 10]);
});

it("should call console.log the correct number of times",
async () => {
    const items = ['a', 'b', 'c'];

    await processInBatches(items, 2, stringProcessor);

    expect(consoleSpy).toHaveBeenCalledTimes(2);
});

it("should return an empty array if an empty array is passed
as input", async () => {
    const items: number[] = [];
    const emptyProcessor = jest.fn().mockResolvedValue([]);

    const result = await processInBatches(items, 2, emptyProcessor);

    expect(result).toEqual([]);
    expect(emptyProcessor).not.toHaveBeenCalled();
    expect(consoleSpy).not.toHaveBeenCalled();
});
});

```

```

PASS src/retryOperation.test.ts
  processInBatches()
    ✓ should return a correct array of results (5 ms)
    ✓ should call console.log the correct number of times (1 ms)
    ✓ should return an empty array if an empty array is passed as input (1 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        0.421 s, estimated 1 s

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – ЛрЗ	Арк.
		Фуріхата Д. В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

3.5. Завдання 5: Тести для processInBatches

```
import { processInBatches } from './t6';

describe('processInBatches()', () => {
  let consoleSpy: jest.SpyInstance;

  beforeEach(() => {
    jest.useFakeTimers();
    consoleSpy = jest.spyOn(console, 'log').mockImplementation(() => {});
  });

  afterEach(() => {
    jest.useRealTimers();
    consoleSpy.mockRestore();
  });

  it('should return a correct array of results', async () => {
    const items = [1, 2, 3, 4, 5];
    const batchSize = 2;

    const processor = async (batch: number[]) => batch.map(num
=> num * 2);

    const result = await processInBatches(items, batchSize,
processor);

    expect(result).toEqual([2, 4, 6, 8, 10]);
  });

  it('should call console.log the correct number of times',
async () => {
    const items = ['a', 'b', 'c'];
    const batchSize = 2;
    const processor = async (batch: string[]) => batch;

    await processInBatches(items, batchSize, processor);

    expect(consoleSpy).toHaveBeenCalledTimes(2);
  });
});
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – ЛрЗ	Арк.
		Фуріхата Д. В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    });

    it('should return an empty array if an empty array is passed
as input', async () => {
        const items: number[] = [];
        const processor = jest.fn().mockResolvedValue([]);

        const result = await processInBatches(items, 2, proces-
sor);

        expect(result).toEqual([]);
        expect(processor).not.toHaveBeenCalled();
        expect(consoleSpy).not.toHaveBeenCalled();
    });
});

```

```

PASS src/processInBatches.test.ts
  processInBatches()
    ✓ should return a correct array of results (5 ms)
    ✓ should call console.log the correct number of times (2 ms)
    ✓ should return an empty array if an empty array is passed as input (2 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        0.419 s, estimated 1 s

```

3.6. Завдання 6: Тести для raceWithTimeout

```

import { raceWithTimeout } from "./t7";

function createDelayedPromise<T>(value: T, delayMs: number): Prom-
ise<T> {
    return new Promise((resolve) => {
        setTimeout(() => resolve(value), delayMs);
    });
}

describe("raceWithTimeout()", () => {
    beforeEach(() => {
        jest.useFakeTimers();
    });

    afterEach(() => {

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – ПрЗ	Арк.
		Фуріхата Д. В.				8
Змн.	Арк.	№ докум.	Підпис	Дата		


```

    jest.useRealTimers();
  });

  it("should resolve with the promise value if it completes before the timeout", async () => {
    const fastPromise = createDelayedPromise("Дані отримано", 100);

    const racePromise = raceWithTimeout(fastPromise, 500);

    await jest.advanceTimersByTimeAsync(100);

    const result = await racePromise;

    expect(result).toBe("Дані отримано");
  });

  it("should reject with a timeout error if the promise takes too long", async () => {
    const slowPromise = createDelayedPromise("Занадто повільно", 1000);

    const racePromise = raceWithTimeout(slowPromise, 500);

    const assertion = expect(racePromise).rejects.toThrow("Operation timed out after 500ms");

    await jest.advanceTimersByTimeAsync(500);

    await assertion;
  });
});

```

```

PASS src/raceWithTimeout.test.ts
  raceWithTimeout()
    ✓ should resolve with the promise value if it completes before the timeout (9 ms)
    ✓ should reject with a timeout error if the promise takes too long (5 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 0.408 s, estimated 1 s

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – ЛрЗ	Арк.
		Фуріхата Д. В.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр3	Арк.
		Фуріхата Д. В.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – ЛрЗ	Арк.
		Фуріхата Д. В.				11
Змн.	Арк.	№ докум.	Підпис	Дата		